2.12

Type Conversions

Implicit conversions • mixed-type arithmetic • explicit type casting

Build on 2.11: control when Java performs integer or floating-point operations.

Learning objectives

By the end of this lesson, students will be able to

- **Explain** why type conversions are needed when mixing int and double values.
- **Identify** when Java performs an **implicit type conversion**.
- **Predict** the result of expressions involving int and double operands.
- **Use type casting** to explicitly convert a value from one data type to another.
- **Avoid unintended integer division** when a floating-point result is required.
- **Explain** what happens when a double is cast to an int.

Unit Overview

Topics Covered in This Lesson

1. **Type Conversions**
   Converting a value from one data type to another, such as int to double.
2. **Implicit Conversions**
   Understanding when Java automatically converts an int to a double.
3. **Mixed-Type Arithmetic**
   Working with expressions that contain both integer and floating-point values.
4. **Type Casting**
   Explicitly converting values using the (type) syntax.
5. **Avoiding Integer Division Errors**
   Using casting or floating-point operands when a fractional result is needed.
6. **Casting double to int**
   Understanding that the fractional part is truncated.

What is a type conversion?

A value changes from one data type to another

A calculation sometimes mixes integer and floating-point values.

```
int numBirths = 1000;
double expectedMales = 0.504 * numBirths;
```

What happens?

0.504 is a double.
numBirths is an int.
Java converts the int value to double for the multiplication.

Type conversion: conversion of one data type to another, such as ***int → double***.

Implicit conversion

Java performs some conversions automatically

Arithmetic operators

If either operand is a double, the other operand is converted to double and floating-point arithmetic is performed.

```
5 + 2.5    → 7.5
5 * 2.0    → 10.0
```

Assignment

Java can convert *the right-side value* to *the left-side type* when the conversion can occur without loss of precision.

int → double: 25 becomes 25.0.

```
double y = 0.5;
Math.pow(y, 3)→ 0.125

int a = Math.pow(y, 3);     //a → 0
double b = Math.pow(y, 3); //b → 0.125
```

Practice: int or double result?

Given: int numItems = 5

```
(numItems + 10) / 2
```

Result: 7

(5 + 10) / 2 = 15 / 2
Both operands are int → integer division.

```
(10 + numItems) / 2.0
```

Result: 7.5

15 / 2.0
One operand is double → floating-point division.

One decimal point can change which kind of division Java performs.

Type casting

The programmer explicitly requests a conversion

Place **(type)** before a value or expression to convert it to the indicated type.

```
int myIntVar = 7;
double x = (double) myIntVar;  // 7.0
```

```
double myDouble = 9.5;
int y = (int) myDouble;          // 9
```

Important

Casting double to int **truncates the fractional part. 9.5 becomes 9; it does not round to 10.**

Casting to get floating-point division

Convert at least one integer operand to double

```
int total = 7;
int count = 2;

double avg = (double) total / count;
// 7.0 / 2 → 3.5
```

Key idea

Without the cast:
7 / 2 → 3

With the cast: keep precision

- **(double) 7 / 2 → 3.5**
- **7 / (double) 2 → 3.5**
- **(double) (7 / 2) → 3.0 X**

Only *one operand* needs to be converted to double.

Common error: unintended integer division

Casting after division is too late

```
double average = (double) (7 / 2);
// 7 / 2 happens first → 3
// then 3 becomes 3.0
```

```
double average = (double) 7 / 2;
// 7 becomes 7.0 first
// 7.0 / 2 → 3.5
```

Remember

Cast before integer division occurs, or use a floating-point literal such as 2.0.

```
int grade1 = 90, grade2 = 70, grade3 = 78;
int total = grade1 + grade2 + grade3;  //total:239
double average = (double) (total / 3);
// total / 3 happens first → 79
// then 3after cast, becomes 79.0
```

```
int grade1 = 90, grade2 = 70, grade3 = 78;
int total = grade1 + grade2 + grade3;  //total:239
double average = (double) total / 3;
// (double)total first → 239.0          //79.666

//double average = (double)(grade1 + grade2 + grade3) / 3;
```

Common error: unintended integer division

Casting after division is too late - lose precision

```
double average = (double) (7 / 2);
// 7 / 2 happens first → 3
// then 3 becomes 3.0
```

```
double average = (double) 7 / 2;
// 7 becomes 7.0 first
// 7.0 / 2 → 3.5
```

Remember

Cast before integer division occurs, or use a floating-point literal such as 2.0.

Fractions in Java formulas

Avoid integer division when the formula requires a fractional constant

```
// Area of a triangle
area = 0.5 * base * height;

// Volume of a sphere
volume = (4.0 / 3.0) * Math.PI * Math.pow(r, 3);
```

Avoid

(1 / 2) * base * height
→ 1 / 2 becomes 0

(4 / 3) * ...
→ 4 / 3 becomes 1

Use 0.5, 1.0 / 2.0, 3.0 / 4.0, 4.0 / 3.0, etc.

Quick practice

Predict the result or conversion

(double) 5 → ?	5.0
(int) 8.9 → ?	8
5 / 2 → ?	2
5 / 2.0 → ?	2.5
(double) 5 / 2 → ?	2.5

Key takeaways

Choose the conversion intentionally

Implicit conversion

Java automatically converts int to double in common mixed-type arithmetic.

Explicit cast

Use (type) when you need to explicitly convert a value.

Watch integer division

int / int gives an integer result, even when assigned to a double.

Need a fractional quotient? Make at least one operand a double before division.